EPICODE
Cyber Security & Ethical Hacking

Relazione tecnica finale

Progetto M1_S4_E2 - Architettura HTTPS con DNS locale e analisi Wireshark

Campo	Valore
Studente	Giovanni D'Angelo
Percorso	Cyber Security & Ethical Hacking
Data esecuzione	04/05/2026
Ambiente	VirtualBox con Kali Linux e Windows 10
Perimetro documentato	Configurazione DNS locale, server HTTPS e cattura traffico HTTPS

Obiettivo: simulare una comunicazione client-server in laboratorio virtuale, risolvere un hostname interno tramite DNS locale e analizzare il traffico HTTPS catturato con Wireshark.

Indice

1. Sintesi operativa

2. Requisiti e architettura realizzata

3. Configurazione rete e indirizzamento IP

4. Configurazione DNS locale

5. Configurazione server HTTPS

6. Verifica da client Windows

7. Analisi traffico con Wireshark

8. Risultati ottenuti

9. Perimetro e scostamenti

10. Conclusione tecnica

1. Sintesi operativa

Nell'esecuzione dell'esercizio ho configurato un ambiente virtuale composto da una macchina Kali Linux con funzione di server e da una macchina Windows 10 con funzione di client. Kali è stata configurata con indirizzo 192.168.32.100, mentre Windows è stato configurato con indirizzo 192.168.32.101.

L'obiettivo operativo della parte documentata è stato creare un servizio DNS locale in grado di risolvere l'hostname epicode.internal verso il server Kali e pubblicare una pagina web raggiungibile tramite HTTPS. Successivamente ho intercettato il traffico con Wireshark, verificando la risoluzione DNS, la presenza dei MAC address sorgente e destinazione e la natura cifrata della comunicazione HTTPS.

La cattura ha mostrato correttamente query e risposta DNS, pacchetti ARP, handshake TCP/TLS e traffico TLS di tipo Application Data. Il contenuto applicativo non è risultato leggibile in chiaro, coerentemente con il funzionamento di HTTPS.

2. Requisiti e architettura realizzata

La traccia del progetto richiede una simulazione client-server in ambiente di laboratorio, con il client Windows che richiede tramite browser una risorsa all'hostname epicode.internal, risolto verso l'indirizzo IP del server Kali.

Elemento	Configurazione richiesta / realizzata
Server	Kali Linux - IP 192.168.32.100
Client	Windows 10 - IP 192.168.32.101
Hostname interno	epicode.internal
Risoluzione DNS	epicode.internal -> 192.168.32.100
Servizio web documentato	HTTPS su porta TCP 443
Strumento di analisi	Wireshark su interfaccia eth1

L'architettura realizzata è quindi composta da due macchine virtuali collegate alla stessa rete interna VirtualBox, denominata EPICODE-LAB. Kali svolge il ruolo di server DNS e server HTTPS; Windows svolge il ruolo di client che effettua la richiesta tramite browser.

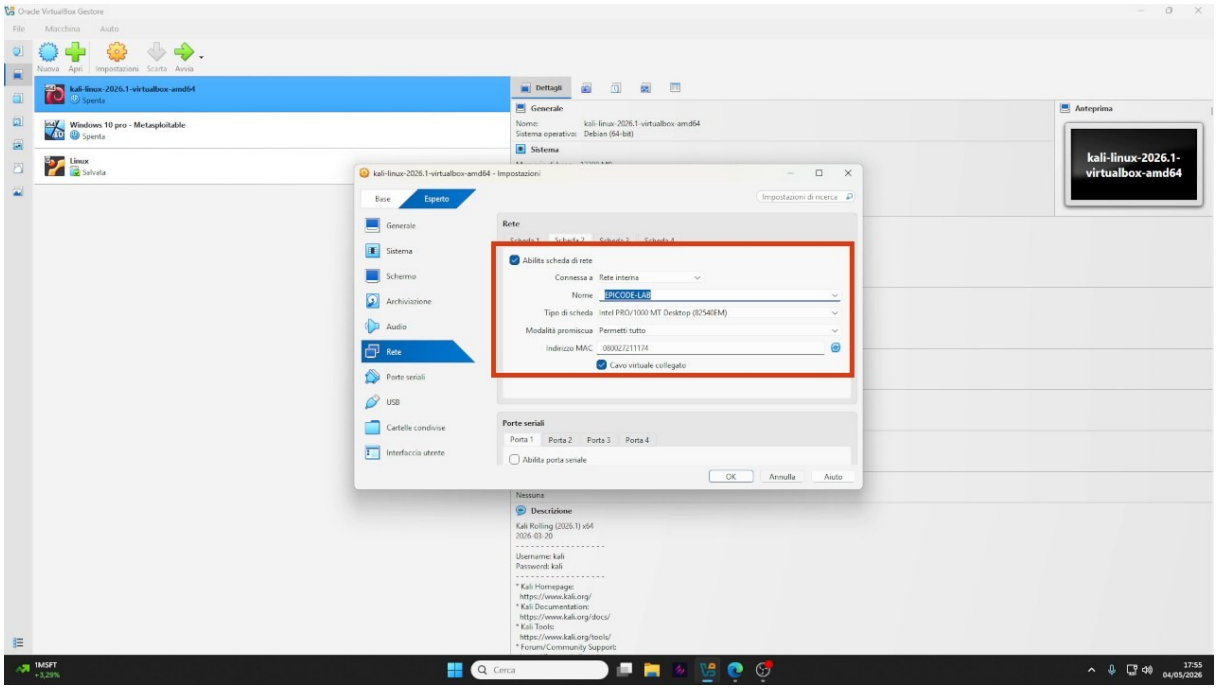

Figura 1 - Configurazione della rete interna VirtualBox EPICODE-LAB.

3. Configurazione rete e indirizzamento IP

3.1 Configurazione lato Kali Linux

Su Kali è stata individuata l'interfaccia di laboratorio eth1, distinta dall'interfaccia NAT. Sull'interfaccia eth1 è stato configurato l'indirizzo statico 192.168.32.100/24, necessario per far funzionare Kali come server raggiungibile dal client Windows.

```
sudo ip addr flush dev eth1
sudo ip addr add 192.168.32.100/24 dev eth1
sudo ip link set eth1 up
ip -br addr
```

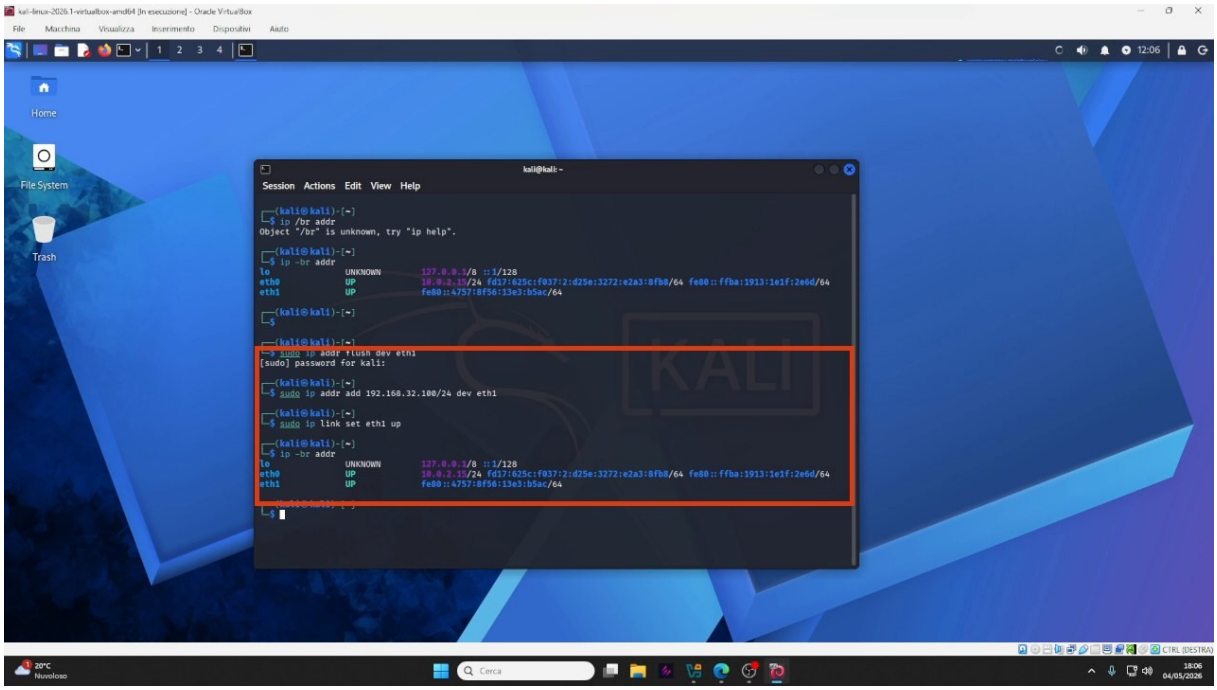

Figura 2 - Configurazione dell'indirizzo IP 192.168.32.100/24 sull'interfaccia eth1 di Kali.

3.2 Configurazione lato Windows

Sul client Windows è stato configurato l'indirizzo 192.168.32.101 con subnet mask 255.255.255.0. Il DNS del client è stato impostato verso Kali, cioè 192.168.32.100, in modo che le richieste per epicode.internal venissero gestite dal DNS locale.

```
Indirizzo IP: 192.168.32.101
Subnet mask: 255.255.255.0
DNS preferito: 192.168.32.100
```

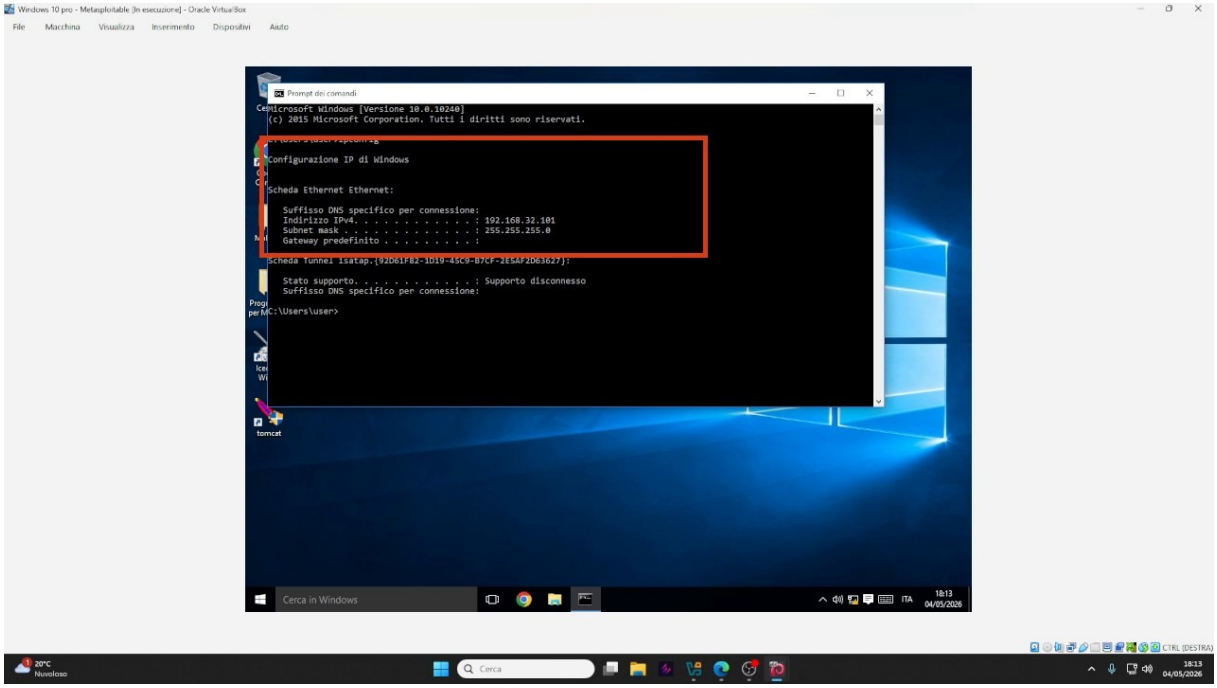

Figura 3 - Verifica ipconfig del client Windows con indirizzo 192.168.32.101.

4. Configurazione DNS locale

Per la risoluzione del nome interno è stato utilizzato dnsmasq. Nel file di configurazione è stato creato il record necessario per associare epicode.internal all'indirizzo 192.168.32.100.

```
address=/epicode.internal/192.168.32.100
```

Dopo il riavvio del servizio, dnsmasq è risultato attivo. La verifica locale tramite nslookup ha confermato che epicode.internal viene risolto correttamente verso 192.168.32.100.

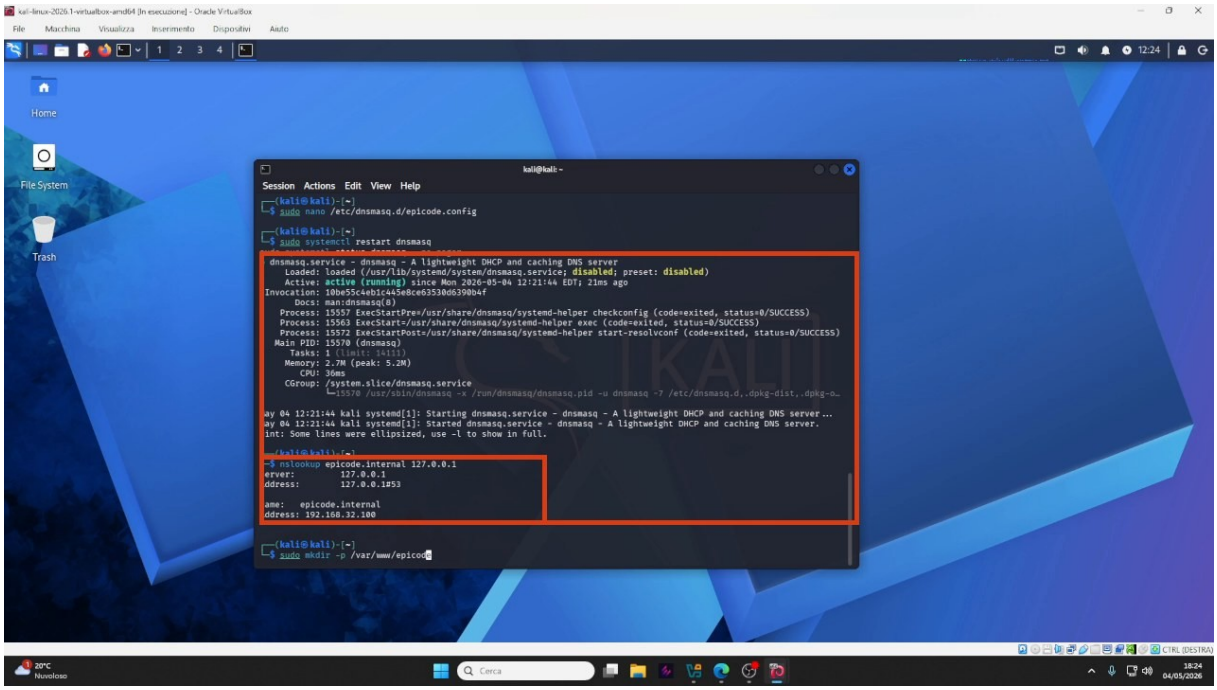

Figura 4 - dnsmasq attivo e verifica nslookup di epicode.internal verso 192.168.32.100.

5. Configurazione server HTTPS

Il servizio web è stato pubblicato tramite Apache2. È stato abilitato il modulo SSL ed è stato configurato un VirtualHost HTTPS sulla porta TCP 443. Per l'ambiente di laboratorio è stato utilizzato un certificato autofirmato associato al nome epicode.internal.

Il servizio Apache è risultato attivo e in ascolto sulla porta 443. Questo conferma che il server Kali era pronto a ricevere richieste HTTPS dal client Windows.

```
sudo systemctl status apache2 --no-pager
sudo ss -lntp | grep ":443"
```

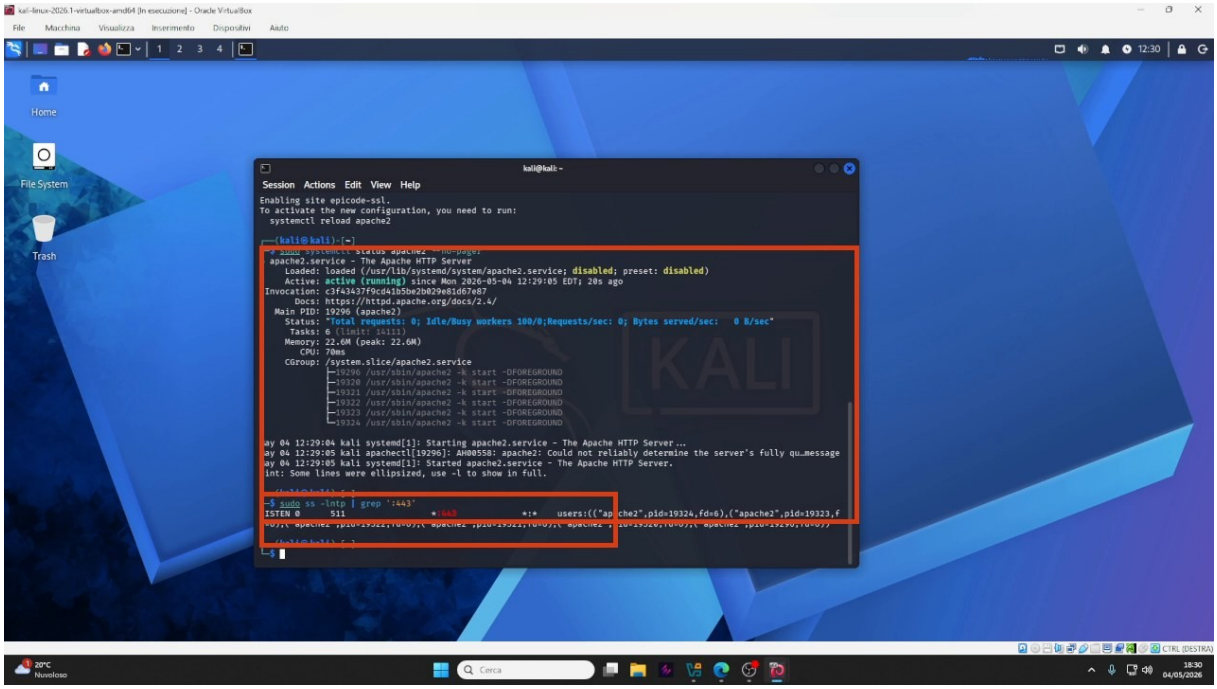

Figura 5 - Apache2 attivo e porta TCP 443 in ascolto sul server Kali.

6. Verifica da client Windows

Dal client Windows è stata aperta la pagina https://epicode.internal. Il browser ha raggiunto correttamente la pagina pubblicata dal server Kali. La dicitura Non sicuro è coerente con l'utilizzo di un certificato autofirmato in ambiente di laboratorio: la connessione resta HTTPS, ma il certificato non è firmato da una Certification Authority riconosciuta dal browser.

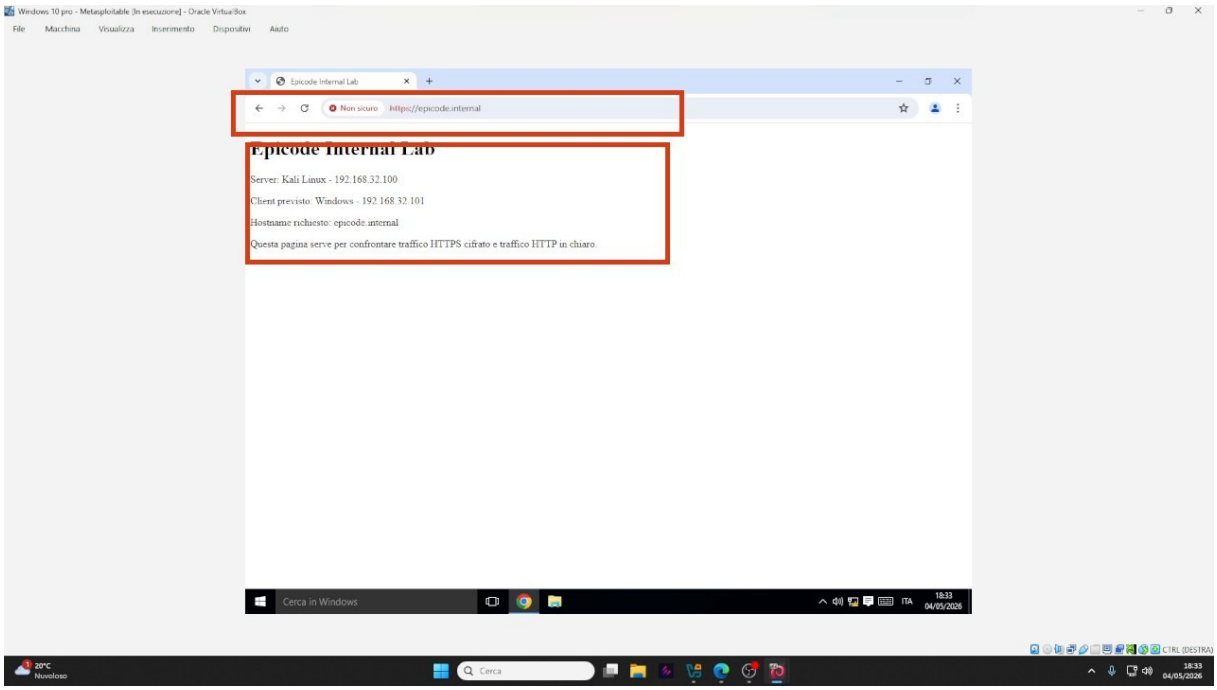

Figura 6 - Pagina HTTPS raggiunta dal client Windows tramite hostname epicode.internal.

7. Analisi traffico con Wireshark

La cattura del traffico è stata eseguita su Kali tramite Wireshark, selezionando l'interfaccia eth1, cioè la scheda di rete collegata alla rete interna EPICODE-LAB. Per concentrarsi sui pacchetti rilevanti è stato utilizzato un filtro composto da ARP, DNS, TLS e traffico TCP sulla porta 443.

```
arp || dns || tls || tcp.port == 443
```

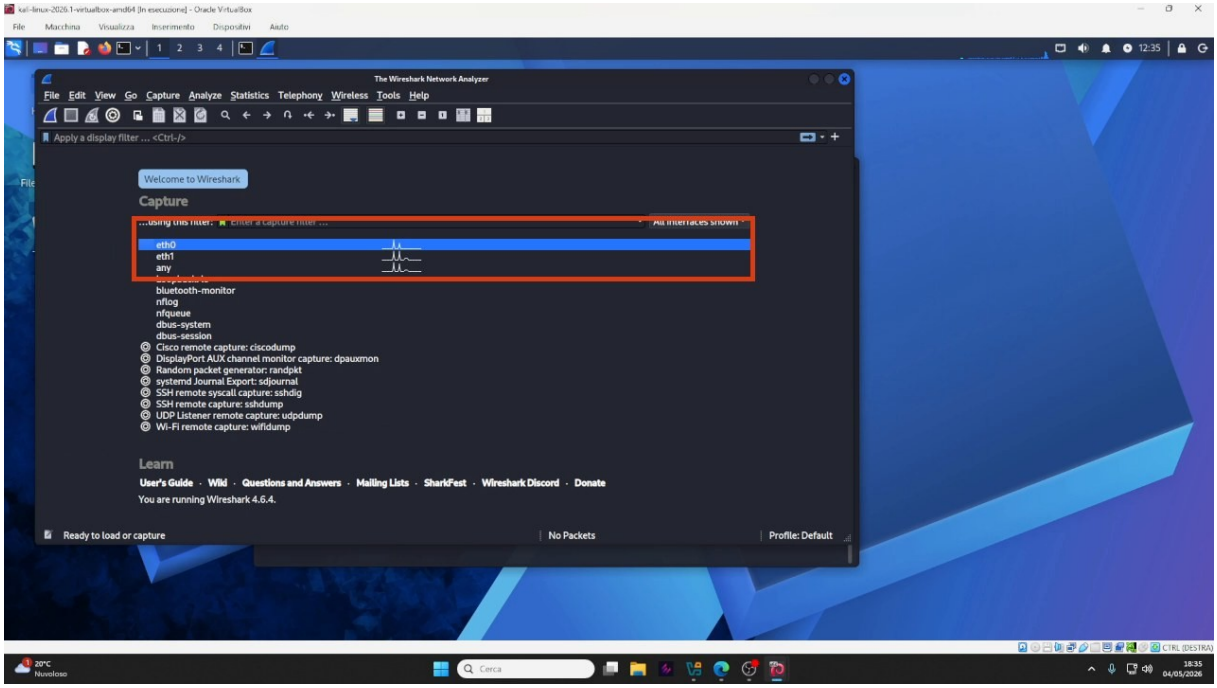

Figura 7 - Selezione dell'interfaccia eth1 in Wireshark per la cattura del traffico di laboratorio.

7.1 Risoluzione DNS osservata

La cattura mostra la query DNS del client 192.168.32.101 verso il server 192.168.32.100 per il record A di epicode.internal. La risposta DNS restituisce correttamente l'indirizzo 192.168.32.100.

Questo passaggio conferma che il browser non si collega direttamente a un IP scritto manualmente, ma usa il nome epicode.internal risolto tramite DNS locale.

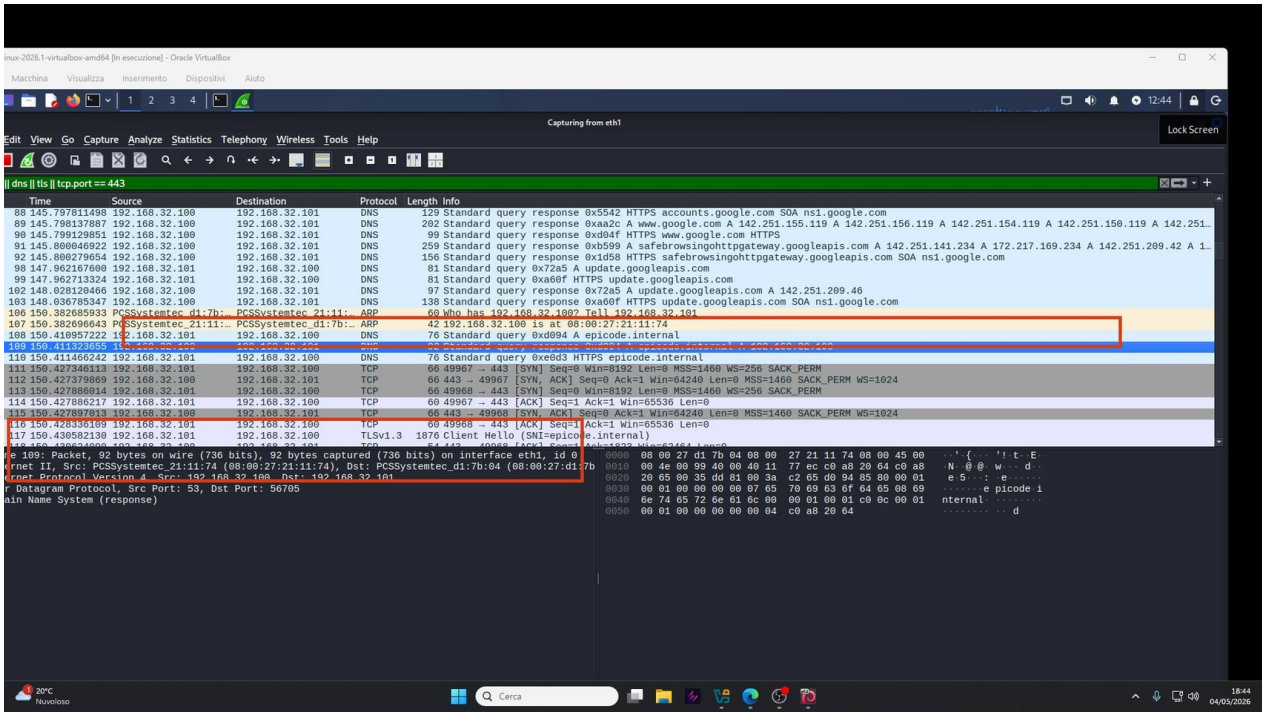

Figura 8 - Risposta DNS: epicode.internal risolto verso 192.168.32.100.

7.2 ARP e MAC address sorgente/destinazione

Prima della comunicazione HTTPS, il client deve conoscere il MAC address del server Kali. Per questo motivo nella cattura compare una richiesta ARP: il client domanda chi possiede 192.168.32.100 e il server risponde comunicando il proprio indirizzo MAC.

Dispositivo	IP	MAC osservato
Client Windows	192.168.32.101	08:00:27:d1:7b:04
Server Kali	192.168.32.100	08:00:27:21:11:74

Nei pacchetti Ethernet II sono visibili i MAC address sorgente e destinazione. Quando il client invia dati al server, il sorgente è il MAC Windows e la destinazione è il MAC Kali; nella risposta avviene l'inverso.

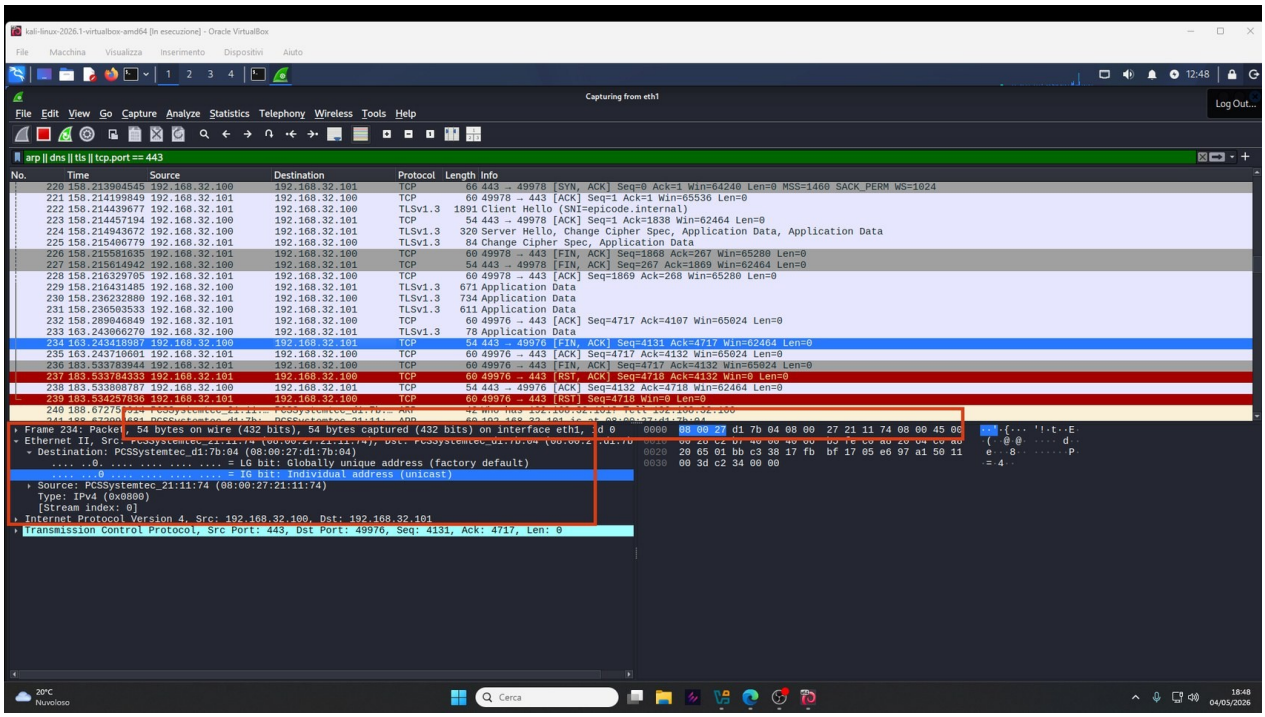

Figura 9 - Pacchetti ARP e dettaglio Ethernet II con MAC address di Kali e Windows.

7.3 Handshake TCP e TLS

Dopo la risoluzione DNS e ARP, il client instaura una connessione TCP verso la porta 443 del server. Nella cattura sono presenti i pacchetti SYN, SYN/ACK e ACK, che rappresentano il classico three-way handshake TCP.

Successivamente parte la negoziazione TLS. Nel pacchetto Client Hello è presente l'indicazione SNI con il nome epicode.internal. Questo dimostra che il client sta avviando una sessione HTTPS verso l'hostname interno configurato.

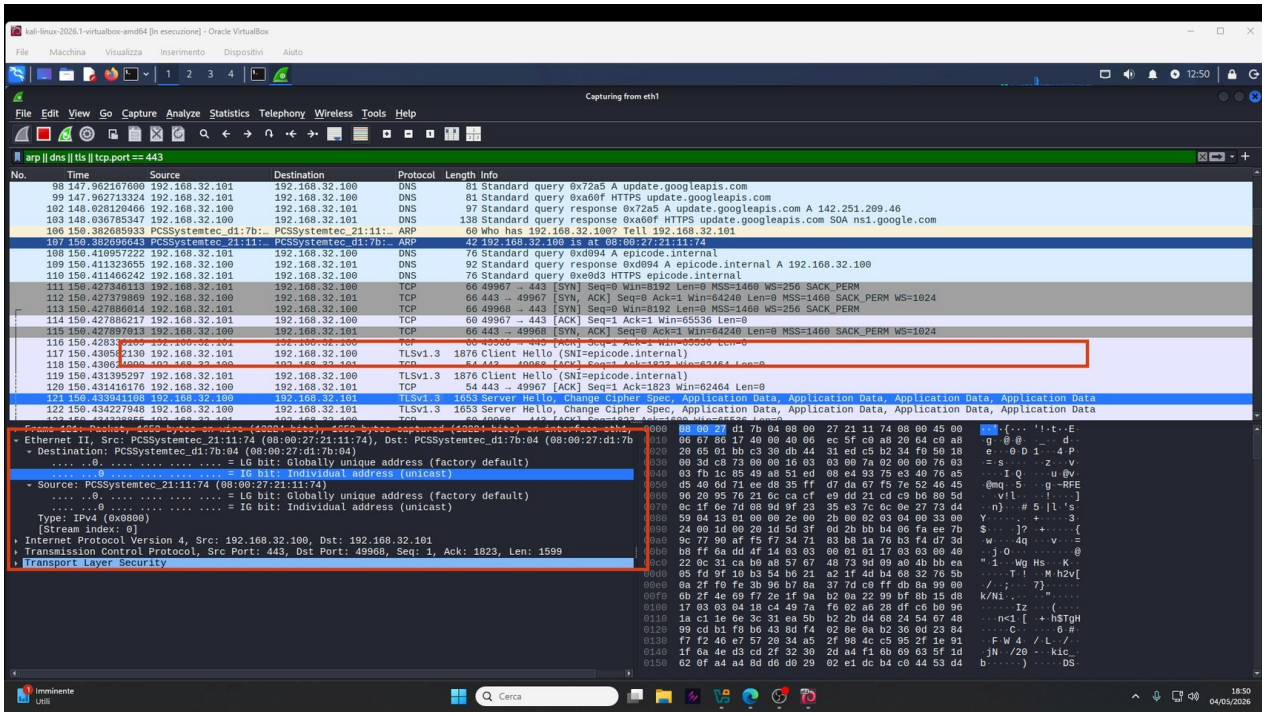

Figura 10 - TLS Client Hello con SNI epicode.internal e collegamento verso la porta 443.

7.4 Contenuto della comunicazione HTTPS

Nei pacchetti TLS successivi si osservano Server Hello, Change Cipher Spec e Application Data. Il dato importante è che Wireshark non mostra una richiesta HTTP leggibile come GET / HTTP/1.1: il contenuto applicativo è cifrato all'interno del canale TLS.

Restano visibili i metadati necessari alla comunicazione, come indirizzi MAC, indirizzi IP e porte TCP, ma il corpo della richiesta e gli header applicativi non sono direttamente leggibili in chiaro.

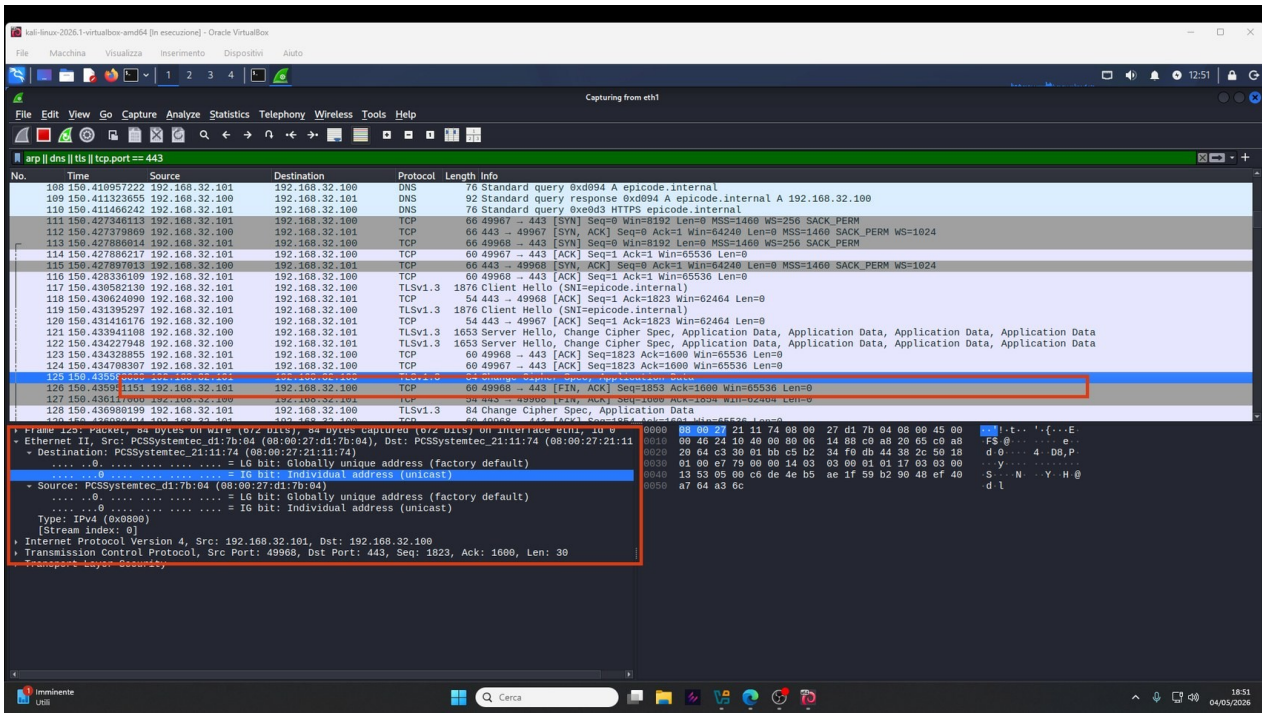

Figura 11 - Pacchetti TLS Application Data: il contenuto applicativo HTTPS risulta cifrato.

8. Risultati ottenuti

Verifica	Risultato
Rete interna VirtualBox	Configurata tramite EPICODE-LAB
IP Kali	192.168.32.100/24 configurato su eth1
IP Windows	192.168.32.101/24 configurato come client
DNS locale	epicode.internal risolto verso 192.168.32.100
Server HTTPS	Apache2 attivo su porta 443
Accesso browser	Pagina https://epicode.internal raggiunta dal client
Cattura DNS	Query e risposta visibili in Wireshark
Cattura ARP	MAC address sorgente e destinazione identificati
Cattura HTTPS	Handshake TLS e Application Data visibili; contenuto cifrato

La prova conferma il corretto funzionamento della parte HTTPS dell'architettura: il nome interno viene risolto dal DNS locale, il server Kali risponde al client Windows e Wireshark consente di osservare i livelli Ethernet, IP, TCP e TLS.

9. Perimetro e scostamenti

La presente relazione documenta in modo completo la configurazione e la verifica della comunicazione HTTPS. La parte relativa alla sostituzione del server HTTPS con un server HTTP non è inclusa tra le evidenze operative della presente esecuzione.

Per questo motivo la relazione non riporta una cattura sperimentale HTTP. La differenza tecnica principale resta comunque motivata dai risultati ottenuti: nel traffico HTTPS il contenuto applicativo viene protetto da TLS e non appare in chiaro in Wireshark; in una comunicazione HTTP non cifrata, invece, gli header e la richiesta applicativa sarebbero leggibili direttamente.

10. Conclusione tecnica

Nell'esecuzione dell'esercizio ho realizzato una comunicazione client-server funzionante tra Windows e Kali, rispettando l'indirizzamento richiesto dalla traccia per la parte HTTPS. Kali è stato configurato come server DNS e server web HTTPS, mentre Windows è stato configurato come client della rete interna.

Le evidenze raccolte con Wireshark confermano che epicode.internal viene risolto correttamente verso 192.168.32.100, che il client 192.168.32.101 comunica con il server tramite la porta 443 e che a livello Ethernet sono identificabili i MAC address sorgente e destinazione.

Il risultato più importante dell'analisi è la conferma pratica del ruolo di HTTPS: pur essendo visibile la struttura della comunicazione, il contenuto applicativo risulta cifrato e quindi non leggibile direttamente. Questo dimostra la differenza di sicurezza introdotta da TLS rispetto a una trasmissione web non cifrata.